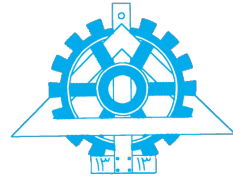




University of Tehran
College of Engineering
School of Electrical and
Computer Engineering



Machine Learning

Final Project Phase 1

Full Name
Sepanta Ghonoodi

Student ID
810102483

February 16, 2026

0. Abstract

This report details the implementation and theoretical analysis of various machine learning models, with a primary focus on optimization, neural network architectures, and the principles of transfer learning. The first half of the assignment covers fundamental statistical methods, including Maximum Likelihood Estimation (MLE), the derivation of stochastic gradient descent updates for Logistic Regression, and the application of Newton's method for regularized linear regression.

The second portion explores the practical aspects of model complexity and generalization. This includes an analysis of the bias-variance trade-off through polynomial regression and ridge regularization, as well as the implementation of a Perceptron algorithm with various data transformations. We also investigate the internal mechanics of neural networks, specifically the representation of functions using computation graphs and the mathematical impact of weight initialization techniques, such as Xavier initialization, on signal stability across deep layers.

The final section focuses on a comparative study of image classification on the CIFAR-10 dataset. We contrast a baseline Multi-Layer Perceptron (MLP) with pre-trained VGG-11 convolutional neural networks, evaluating the performance gap between models trained from scratch, those using frozen feature extractors, and those undergoing full fine-tuning. This analysis highlights the importance of spatial inductive bias and pre-trained representations in modern deep learning tasks.

Contents

Abstract	i
1 Question 1: Maximum Likelihood Estimation	1
1.1 Derivation of the Maximum Likelihood Estimator	1
2 Question 2: Logistic Regression with One Feature	2
2.1 Derivation of the SGD Update Rule	2
3 Question 3: L2-Regularized Linear Regression with Newton's Method	3
3.1 Proof of Convergence in One Step	4
4 Question 4: Polynomial Regression and Ridge Regularization	5
4.1 Part (A): Model Selection on Dataset A	6
4.2 Part (B): Bias-Variance Trade-off on Dataset B	7
4.2.1 Effect of Polynomial Degree	7
4.2.2 Effect of Ridge Regularization	8
4.3 Discussion	8
5 Question 5: Perceptron	9
5.1 Perceptron Update Step	9
5.2 Convergence Analysis	10
5.3 Valid Data Transformations	10
5.4 Weight Vector for $[x, y, x^2 + y^2, 1]$	11
5.5 Alternative Transformation	11
6 Question 6: Neural Network Representations	12
6.1 Analysis of Computation Graphs	12
7 Question 7: Loss function	14
7.1 Multi-class Likelihood vs. Cross-Entropy	14
7.2 Error Function with Noisy Labels	15
7.3 Error Function for Targets in $\{-1, 1\}$	16
8 Question 8: Stochastic Stability and the "Dead ReLU"	16
8.1 1. Distribution Proof	17
8.2 2. Activation Probability	18
8.3 3.	18
8.4 4. The Zero-Gradient Proof	19
9 Question 9: Variance Preservation and Xavier Initialization	19
9.1 1. Symmetric Constraints	20
9.2 The Glorot Synthesis	20
9.3 Depth-Induced Variance	21
9.4 The Stability Bound	21
9.5 Numerical Analysis	21
10 Question 10: Transfer Learning with Pre-trained Networks	22
10.1 Results	22

10.2	Discussion	22
10.2.1	Convergence and Stability	22
10.2.2	2. Generalization Gap and Spatial Inductive Bias	23

1. Question 1: Maximum Likelihood Estimation

Let $x_1, x_2, \dots, x_n$ be independent samples from the following distribution:

$$P(x|\theta) = \theta x^{-\theta-1}, \text{ where } \theta > 1, x \geq 1.$$

Find the maximum likelihood estimator of θ .

1.1 Derivation of the Maximum Likelihood Estimator

To find the Maximum Likelihood Estimator (MLE) for the parameter θ , we first define the likelihood function $L(\theta)$ for the independent samples $x_1, \dots, x_n$.

The probability density function is given by:

$$P(x|\theta) = \theta x^{-\theta-1}$$

The likelihood function is the product of the individual probabilities:

$$L(\theta) = \prod_{i=1}^n P(x_i|\theta) = \prod_{i=1}^n \theta x_i^{-\theta-1}$$

Simplifying the product:

$$L(\theta) = \theta^n \left(\prod_{i=1}^n x_i \right)^{-\theta-1}$$

To make the derivation easier, we take the natural logarithm to find the log-likelihood function, $\ell(\theta) = \ln L(\theta)$:

$$\ell(\theta) = \ln \left(\theta^n \left(\prod_{i=1}^n x_i \right)^{-\theta-1} \right)$$

Using logarithm properties:

$$\ell(\theta) = n \ln(\theta) + (-\theta - 1) \sum_{i=1}^n \ln(x_i)$$

$$\ell(\theta) = n \ln(\theta) - (\theta + 1) \sum_{i=1}^n \ln(x_i)$$

To maximize the log-likelihood, we take the derivative with respect to θ and set it to zero:

$$\frac{d\ell}{d\theta} = \frac{n}{\theta} - \sum_{i=1}^n \ln(x_i)$$

Setting the derivative to zero to find the critical point:

$$\frac{n}{\hat{\theta}} - \sum_{i=1}^n \ln(x_i) = 0$$

Solving for $\hat{\theta}$:

$$\begin{aligned} \frac{n}{\hat{\theta}} &= \sum_{i=1}^n \ln(x_i) \\ \hat{\theta}_{\text{MLE}} &= \frac{n}{\sum_{i=1}^n \ln(x_i)} \end{aligned}$$

2. Question 2: Logistic Regression with One Feature

We are given a sample in which each point has only one feature. Consider a binary classification problem in which sample values $x \in \mathbb{R}$ are drawn randomly from two different class distributions. The first class, with label $y = 0$, has its mean to the left of the mean of the second class, with label $y = 1$. We will use a modified version of logistic regression to classify these data points.

We model the posterior probability at a test point $z \in \mathbb{R}$ as

$$P(y = 1|z) = s(z - \alpha),$$

where $\alpha \in \mathbb{R}$ is the sole parameter we are trying to learn and

$$s(\gamma) = \frac{1}{1 + e^{-\gamma}}$$

is the logistic function. The decision boundary is $z = \alpha$ (because $s(z) = \frac{1}{2}$ there).

We will learn the parameter α by performing gradient descent on the logistic loss function (a.k.a. cross-entropy). That is, for a data point with label $y \in \{0, 1\}$ we find the α that minimizes

$$J(\alpha) = -y \ln s(x - \alpha) - (1 - y) \ln(1 - s(x - \alpha)).$$

Derive the stochastic gradient descent update for J with step size $\epsilon > 0$, given a sample value x and a label y .

Hint: feel free to use s as an abbreviation for $s(x - \alpha)$.

2.1 Derivation of the SGD Update Rule

To derive the stochastic gradient descent (SGD) update rule, we need to calculate the gradient of the loss function $J(\alpha)$ with respect to the parameter α .

The loss function is given by:

$$J(\alpha) = -y \ln(s) - (1 - y) \ln(1 - s)$$

where $s = s(x - \alpha) = \frac{1}{1 + e^{-(x - \alpha)}}$.

We apply the chain rule to find $\frac{\partial J}{\partial \alpha}$:

$$\frac{\partial J}{\partial \alpha} = \frac{\partial J}{\partial s} \cdot \frac{\partial s}{\partial \alpha}$$

Compute $\frac{\partial J}{\partial s}$

$$\begin{aligned} \frac{\partial J}{\partial s} &= -\frac{y}{s} - (1 - y) \left(\frac{-1}{1 - s} \right) \\ \frac{\partial J}{\partial s} &= -\frac{y}{s} + \frac{1 - y}{1 - s} \end{aligned}$$

Finding a common denominator:

$$\frac{\partial J}{\partial s} = \frac{-y(1 - s) + s(1 - y)}{s(1 - s)} = \frac{-y + ys + s - ys}{s(1 - s)} = \frac{s - y}{s(1 - s)}$$

Compute $\frac{\partial s}{\partial \alpha}$ Recall that the derivative of the sigmoid function $\sigma(z)$ with respect to its argument is $\sigma(z)(1 - \sigma(z))$. Here, the argument is $(x - \alpha)$. By the chain rule:

$$\begin{aligned} \frac{\partial s}{\partial \alpha} &= s(1 - s) \cdot \frac{\partial}{\partial \alpha}(x - \alpha) \\ \frac{\partial s}{\partial \alpha} &= s(1 - s) \cdot (-1) \end{aligned}$$

Combine to find $\frac{\partial J}{\partial \alpha}$

$$\begin{aligned} \frac{\partial J}{\partial \alpha} &= \left(\frac{s - y}{s(1 - s)} \right) \cdot (-s(1 - s)) \\ \frac{\partial J}{\partial \alpha} &= -(s - y) = y - s \end{aligned}$$

Alternatively, substituting $s(x - \alpha)$ back in:

$$\nabla_{\alpha} J = y - s(x - \alpha)$$

The SGD Update Rule The standard gradient descent update rule for a parameter α with learning rate (step size) ϵ is:

$$\alpha_{new} \leftarrow \alpha_{old} - \epsilon \frac{\partial J}{\partial \alpha}$$

Substituting our gradient:

$$\begin{aligned} \alpha_{new} &\leftarrow \alpha - \epsilon(y - s(x - \alpha)) \\ \alpha_{new} &\leftarrow \alpha + \epsilon(s(x - \alpha) - y) \end{aligned}$$

Thus, the stochastic gradient descent update is:

$$\alpha \leftarrow \alpha + \epsilon(s(x - \alpha) - y)$$

3. Question 3: L2-Regularized Linear Regression with Newton's Method

Recall that the objective function for L2-regularized linear regression is

$$J(w) = ||Xw - y||_2^2 + \lambda ||w||_2^2,$$

where X is the design matrix (the rows of X are the data points). The global minimizer of J is given by

$$w^* = (X^\top X + \lambda I)^{-1} X^\top y$$

Consider running Newton's method to minimize J . Let w_0 be an arbitrary initial guess for Newton's method. Show that w_1 , the value of the weights after one Newton step, is equal to w^* .

3.1 Proof of Convergence in One Step

Newton's method update rule for minimizing a function $J(w)$ is given by:

$$w_{new} = w_{old} - H^{-1} \nabla J(w_{old})$$

where $\nabla J(w)$ is the gradient and H is the Hessian matrix (second derivative) of $J(w)$.

Compute the Gradient $\nabla J(w)$ The objective function is:

$$J(w) = (Xw - y)^\top (Xw - y) + \lambda w^\top w$$

Expanding the terms:

$$J(w) = w^\top X^\top X w - 2y^\top X w + y^\top y + \lambda w^\top w$$

Taking the derivative with respect to w :

$$\nabla J(w) = 2X^\top X w - 2X^\top y + 2\lambda w$$

$$\nabla J(w) = 2(X^\top X + \lambda I)w - 2X^\top y$$

Compute the Hessian H The Hessian is the derivative of the gradient with respect to w :

$$H = \nabla^2 J(w) = \frac{\partial}{\partial w} (2(X^\top X + \lambda I)w - 2X^\top y)$$

Since the gradient is linear in w , the Hessian is a constant matrix:

$$H = 2(X^\top X + \lambda I)$$

Apply the Newton Update Let the initial guess be w_0 . The update for w_1 is:

$$w_1 = w_0 - H^{-1} \nabla J(w_0)$$

Substitute H and $\nabla J(w_0)$:

$$w_1 = w_0 - (2(X^\top X + \lambda I))^{-1} (2(X^\top X + \lambda I)w_0 - 2X^\top y)$$

Factor out the scalar 2 and simplify:

$$w_1 = w_0 - (2(X^\top X + \lambda I))^{-1} 2((X^\top X + \lambda I)w_0 - X^\top y)$$

$$w_1 = w_0 - (X^\top X + \lambda I)^{-1} ((X^\top X + \lambda I)w_0 - X^\top y)$$

Distribute the inverse matrix:

$$w_1 = w_0 - [(X^\top X + \lambda I)^{-1}(X^\top X + \lambda I)w_0 - (X^\top X + \lambda I)^{-1}X^\top y]$$

Since $A^{-1}A = I$:

$$w_1 = w_0 - [Iw_0 - (X^\top X + \lambda I)^{-1}X^\top y]$$

$$w_1 = w_0 - w_0 + (X^\top X + \lambda I)^{-1}X^\top y$$

Conclusion

$$w_1 = (X^\top X + \lambda I)^{-1}X^\top y$$

We observe that this is exactly the closed-form solution w^* :

$$w_1 = w^*$$

Thus, Newton's method converges to the global minimizer in a single step regardless of the initial guess w_0 . This occurs because the objective function $J(w)$ is quadratic (convex), and Newton's method approximates the function with a quadratic form—which in this case is exact.

4. Question 4: Polynomial Regression and Ridge Regularization

4.1 Part (A): Model Selection on Dataset A

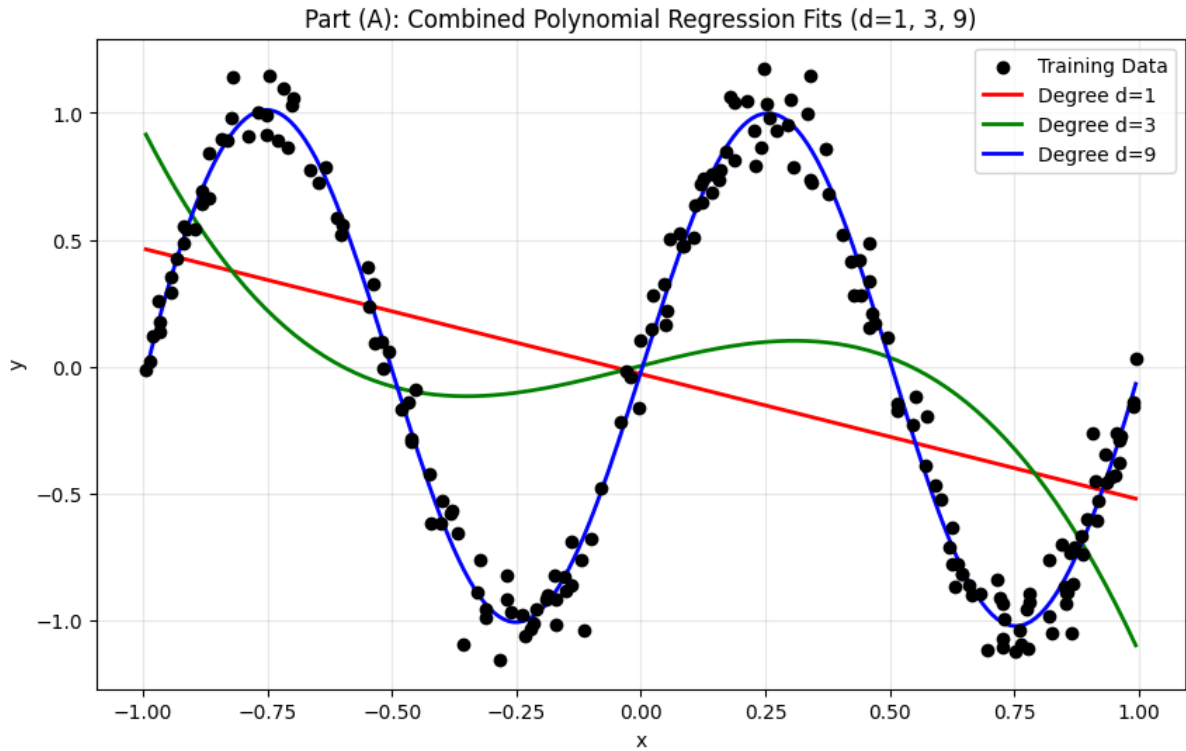


Figure 1: Polynomial regression fits for degrees $d = 1, 3, 9$ on Dataset A. The blue line ($d = 9$) captures the underlying periodic pattern, while red ($d = 1$) and green ($d = 3$) fail to capture the curvature.

Based on the visual inspection of the fitted curves in the figure above:

- **Underfitting:** The models with degrees $d = 1$ (red line) and $d = 3$ (green line) clearly underfit the data. The linear model ($d = 1$) fails completely to capture the periodic nature of the data. The cubic model ($d = 3$) captures some curvature but lacks the flexibility to model the multiple peaks and valleys present in the true function.
- **Complexity of $d = 9$:** The model with degree $d = 9$ (blue line) fits the data points very well. Relative to the visible pattern in the points, it does not show signs of unnecessary complexity. It tracks the underlying sine-like wave smoothly without exhibiting high-frequency oscillations or "wiggles" between data points that would characterize overfitting.

4.2 Part (B): Bias-Variance Trade-off on Dataset B

4.2.1 Effect of Polynomial Degree

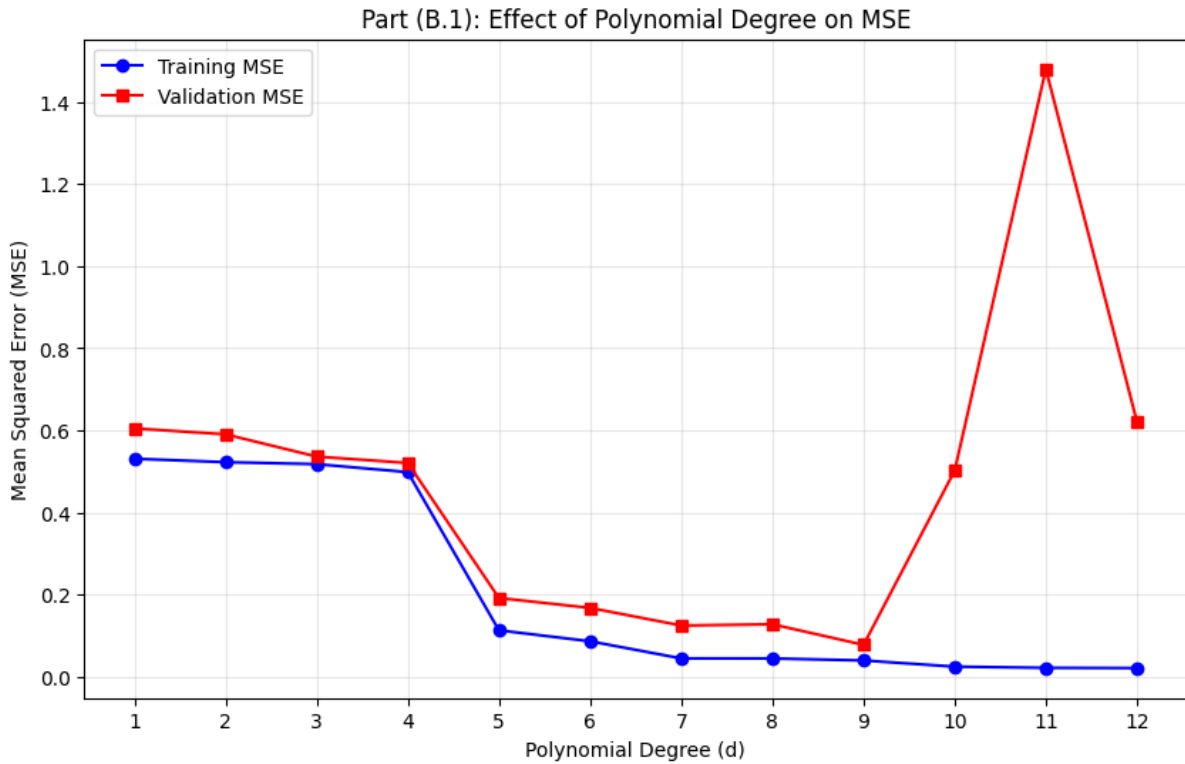


Figure 2: Training and Validation MSE as a function of polynomial degree d .

The plot of MSE versus polynomial degree reveals the classic bias-variance trade-off:

- **Underfitting Range:** Degrees $d = 1$ to $d = 4$ show high error for both training (blue) and validation (red). This indicates underfitting (high bias), where the model is too simple to capture the underlying relationship.
- **Overfitting Range:** For degrees $d \geq 10$, the validation MSE spikes dramatically while the training MSE remains near zero. This indicates overfitting (high variance), where the model fits the training noise rather than the signal.
- **Best Trade-off:** The best trade-off occurs in the range of $d = 7$ to $d = 9$, where the validation MSE reaches its minimum. Specifically, $d = 9$ appears to offer a very low validation error before the instability of higher degrees sets in.

4.2.2 Effect of Ridge Regularization

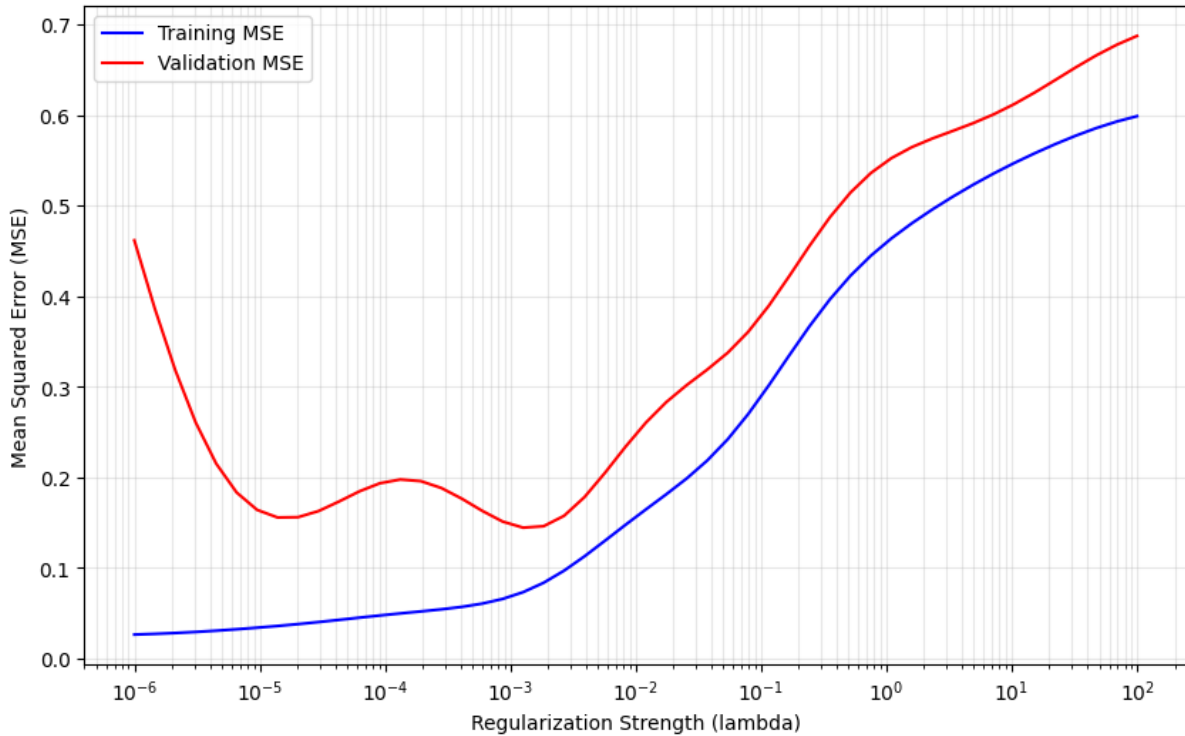


Figure 3: Training and Validation MSE as a function of regularization strength λ (log scale).

Using a fixed high-degree polynomial, we observe the effect of the regularization parameter λ :

- **Effect of Increasing λ :** Initially, increasing λ reduces the validation error significantly by penalizing large weights and reducing model complexity (mitigating overfitting). However, as λ becomes too large, the model is constrained too heavily, causing both training and validation errors to rise (transitioning to underfitting).
- **Optimal Range:** The validation MSE reaches its minimum approximately between $\lambda = 10^{-3}$ and $\lambda = 10^{-2}$.
- **Extremes:**
 - At very small λ ($< 10^{-4}$), the model behaves like unregularized least squares, resulting in high validation error due to overfitting.
 - At very large λ (> 1), the penalty dominates the loss function, forcing weights toward zero. This results in a trivial model (underfitting) with high error on both datasets.

4.3 Discussion

Comparing the observations, we see that model complexity determines the capacity to fit data. In Part (A), a higher degree ($d = 9$) was necessary to capture the complex structure. In Part (B),

however, blindly increasing complexity ($d = 11, 12$) led to overfitting. Ridge regularization provides a solution by allowing us to use a high-complexity model (high d) while using λ to control the effective complexity, preventing the weights from exploding and maintaining generalization performance.

5. Question 5: Perceptron

Consider a dataset with 6 points on a 2D coordinate grid, where each point belongs to one of two classes:

- Negative class: $[-1, 0], [1, 0], [0, 1]$
 - Positive class: $[-2, 0], [2, 0], [0, 2]$
- (a) Suppose we run the perceptron algorithm with the initial weight vector set to $[0, 5]$. What is the updated weight vector after processing the data point $[0, 1]$?
 - (b) How many iterations of the perceptron algorithm will run before the algorithm converges? Processing one data point counts as one iteration. If the algorithm never converges, write ∞ .
 - (c) Which of the following data transformations will cause the perceptron algorithm to converge, when run on the transformed data? Select all that apply.
 - $[x, y] \rightarrow [y, x]$
 - $[x, y] \rightarrow [x, y, 1]$
 - $[x, y] \rightarrow [x, y, x^2, 1]$
 - $[x, y] \rightarrow [x, y, x^2 + y^2]$
 - None of the above.
 - (d) Suppose we transform $[x, y]$ to $[x, y, x^2 + y^2, 1]$, and pass the transformed data points into the perceptron. Write one possible weight vector that the perceptron algorithm may converge to.
 - (e) Construct another transformation (not equal to the ones above) that will allow the perceptron algorithm to converge. Hint: The transformation $[x, y] \rightarrow [x, y, x^2 + y^2, 1]$ allows the perceptron algorithm to converge. Fill in the blank: $[x, y] \rightarrow [x, y, _, 1]$.

5.1 Perceptron Update Step

We are given:

- Initial weight vector: $w_{old} = [0, 5]$.

- Data point: $x = [0, 1]$.
- True label: The point $[0, 1]$ belongs to the Negative class, so $y = -1$.

First, we compute the prediction:

$$\hat{y} = \text{sign}(w_{old} \cdot x) = \text{sign}([0, 5] \cdot [0, 1]) = \text{sign}(0 + 5) = \text{sign}(5) = +1$$

Since the prediction ($\hat{y} = +1$) does not match the true label ($y = -1$), we perform an update. The perceptron update rule is:

$$w_{new} = w_{old} + y \cdot x$$

Substituting the values:

$$w_{new} = [0, 5] + (-1)[0, 1]$$

$$w_{new} = [0, 5] + [0, -1]$$

$$w_{new} = [0, 4]$$

Answer: $[0, 4]$

5.2 Convergence Analysis

We analyze the geometry of the dataset:

- The negative points are at distance 1 from the origin ($r = 1$).
- The positive points are at distance 2 from the origin ($r = 2$).

The negative class is "surrounded" by the positive class (or specifically, the origin is surrounded by negatives, which are surrounded by positives). The convex hull of the negative class lies strictly inside the convex hull of the positive class.

A standard Perceptron defines a linear decision boundary (a line in 2D). It is geometrically impossible to separate these two concentric sets with a single straight line. Since the dataset is not linearly separable, the Perceptron algorithm will cycle indefinitely.

Answer: ∞

5.3 Valid Data Transformations

We evaluate which transformation makes the data linearly separable:

- $[x, y] \rightarrow [y, x]$: Still linear 2D, not separable.
- $[x, y] \rightarrow [x, y, 1]$: Adds a bias, but the decision boundary is still a line in the original space. Not separable.

- $[x, y] \rightarrow [x, y, x^2, 1]$: This works. We can find weights such that the decision boundary is parabolic. For example, weights $w = [0, 2, 1, -3]$ create a boundary $2y + x^2 - 3 > 0$.
 - Pos $(\pm 2, 0) \rightarrow 0 + 4 - 3 = 1 > 0$ (Correct)
 - Pos $(0, 2) \rightarrow 4 + 0 - 3 = 1 > 0$ (Correct)
 - Neg $(\pm 1, 0) \rightarrow 0 + 1 - 3 = -2 < 0$ (Correct)
 - Neg $(0, 1) \rightarrow 2 + 0 - 3 = -1 < 0$ (Correct)
- $[x, y] \rightarrow [x, y, x^2 + y^2]$: This transformation lacks a bias term (the constant '1'). Without a bias, the separating plane in 3D must pass through the origin. Since the negative points map to $z = 1$ and positive points map to $z = 4$, a plane through the origin cannot separate them (a ray from the origin passes through the negative cluster to reach the positive cluster).

Selected Option:

- $[x, y] \rightarrow [x, y, x^2, 1]$

5.4 Weight Vector for $[x, y, x^2 + y^2, 1]$

The transformation maps inputs to $[x, y, z, 1]$ where $z = x^2 + y^2$.

- Negative points have distance $r = 1 \implies z = 1$.
- Positive points have distance $r = 2 \implies z = 4$.

We need a weight vector $w = [w_1, w_2, w_3, w_4]$ such that $w \cdot \phi > 0$ for positives and $w \cdot \phi < 0$ for negatives. We can ignore x and y and rely solely on the squared radius z . Let $w_1 = 0, w_2 = 0$. We need $w_3(4) + w_4 > 0$ and $w_3(1) + w_4 < 0$.

Let $w_3 = 1$ and $w_4 = -2.5$.

- Pos: $1(4) - 2.5 = 1.5 > 0$ (Correct)
- Neg: $1(1) - 2.5 = -1.5 < 0$ (Correct)

Answer: $[0, 0, 1, -2.5]$ (Other valid solutions exist, e.g., $[0, 0, 1, -3]$)

5.5 Alternative Transformation

We need a feature that distinguishes the "inner" points from the "outer" points, similar to $x^2 + y^2$ or x^2 , but distinct from the list in Part 3.

The L_1 norm (Manhattan distance) works perfectly here:

- Negative points: $|x| + |y| = 1$

- Positive points: $|x| + |y| = 2$

By using $|x| + |y|$ as the feature, the classes become linearly separable (e.g., threshold at 1.5).

Another valid option is y^2 . Since x^2 was listed in Part 3 as a valid option, y^2 is a symmetric alternative that also enables separability (by using the x coordinate to handle the x-axis points and y^2 to handle the y-axis points).

Answer: $|x| + |y|$

6. Question 6: Neural Network Representations

You are given a number of functions (a-h) of a single variable, x , which are graphed above. The computation graphs shown below will start off simple and get more complex, building up to neural networks.

- For each computation graph, indicate which of the functions above it is able to represent.
- For those that can, write out appropriate values of the weights and biases (for computation graphs G_1 to G_5).

Functions: (a) $2x$ (b) $4x - 5$ (c) $\begin{cases} 2x - 5 & x \geq 2.5 \\ 0 & x < 2.5 \end{cases}$ (d) $\begin{cases} -2x - 5 & x \leq -2.5 \\ 0 & x > -2.5 \end{cases}$ (e) $\begin{cases} -x + 3 & x \geq 2 \\ 1 & x < 2 \end{cases}$ (f) $\begin{cases} 3 & x \leq 0 \\ 3 - x & 0 < x \leq 3 \\ 0 & x > 3 \end{cases}$ (g) $\log(x)$ (h) $\begin{cases} 0.5x & x \leq 0 \\ 0 & 0 < x \leq 3 \\ 3x - 9 & x > 3 \end{cases}$

Computation Graphs: $G_1 : x \xrightarrow{w} * \rightarrow o$ $G_2 : x \xrightarrow{w} * \xrightarrow{b} + \rightarrow o$

$G_3 : x \xrightarrow{w} * \xrightarrow{b} + \xrightarrow{\text{Relu}} o$ $G_4 : x \xrightarrow{w_1} * \xrightarrow{b_1} + \xrightarrow{w_2} * \xrightarrow{b_2} + \rightarrow o$

$G_5 : x \xrightarrow{w_1} * \xrightarrow{b_1} + \xrightarrow{\text{Relu}} * \xrightarrow{w_2} + \xrightarrow{b_2} o$ $G_6 : \text{Sum of two ReLUs in sequence}$

$G_7 : \text{Parallel sum of two ReLUs}$

6.1 Analysis of Computation Graphs

G_1

- Structure: $o = w \cdot x$
- Matches Function (a): $2x$. This is a linear function passing through the origin.
- Weights: Set $w = 2$.

2. Graph G_2

- Structure: $o = w \cdot x + b$
- Matches Function (b): $4x - 5$. This is a standard linear (affine) function.
- Weights: Set $w = 4, b = -5$.

G_3

- Structure: $o = \text{ReLU}(w \cdot x + b) = \max(0, wx + b)$
- Matches Function (c): This function is 0 for $x < 2.5$ and linear $2x - 5$ for $x \geq 2.5$. This matches the shape of a ReLU.
 - Weights: Set $w = 2, b = -5$.
 - Check: $\max(0, 2x - 5)$. If $x = 2.5$, $2(2.5) - 5 = 0$.
- Matches Function (d): This function is 0 for $x > -2.5$ and linear $-2x - 5$ for $x \leq -2.5$.
 - Weights: Set $w = -2, b = -5$.
 - Check: $\max(0, -2x - 5)$. If $x = -3$, $-2(-3) - 5 = 1 > 0$. If $x = 0$, $-5 < 0 \rightarrow 0$.

G_4

- Structure: $o = w_2(w_1x + b_1) + b_2 = (w_1w_2)x + (w_2b_1 + b_2)$.
- Analysis: This is composed of linear operations, so the result is still a single linear function $Ax + B$.
- Matches Function (b): $4x - 5$.
- Weights: There are multiple valid solutions.
 - Let $w_2 = 1, b_2 = 0$.
 - Then $w_1 = 4, b_1 = -5$.

G_5

- Structure: $o = w_2 \cdot \text{ReLU}(w_1x + b_1) + b_2$.
- Matches Function (e): The function is constant ($y = 1$) for $x < 2$ and linear ($y = -x + 3$) for $x \geq 2$.
 - The constant region is determined by the bias b_2 when the ReLU is inactive (0). Thus, $b_2 = 1$.
 - The "hinge" is at $x = 2$, where $w_1x + b_1 = 0$. If we choose $w_1 = 1$, then $1(2) + b_1 = 0 \Rightarrow b_1 = -2$.
 - For $x > 2$, the ReLU is active ($x - 2$). The output is $w_2(x - 2) + 1$.
 - We want this to equal $-x + 3$.
 - $w_2x - 2w_2 + 1 = -x + 3$.
 - Comparing x terms: $w_2 = -1$.
 - Weights: $w_1 = 1, b_1 = -2, w_2 = -1, b_2 = 1$.

G_6 and G_7

- These graphs sum multiple ReLU units, allowing them to represent piecewise linear functions with multiple slope changes. They can represent functions (f) and (h), but the question only requests weights for G_1 to G_5 . Function (g) $\log(x)$ is non-linear and cannot be exactly represented by a finite composition of ReLUs.

7. Question 7: Loss function

In this question, we explore the connection between maximum likelihood and cross-entropy in neural network classification, considering extensions to noisy labels and alternative target representations.

- Show that maximizing the likelihood for a multi-class neural network model in which the network outputs have the interpretation $y_k(x, w) = p(t_k = 1|x)$ is equivalent to minimizing the cross-entropy error function $E(w) = -\sum_{n=1}^N \sum_{k=1}^K t_{kn} \ln y_k(x_n, w)$ (where K is the number of mutually exclusive classes and t_{kn} is the k th component of the one-of- K target vector for data point n).
- Consider a binary classification problem in which the target values are $t \in \{0, 1\}$, with a network output $y(x, w)$ that represents $p(t = 1|x)$, and suppose that there is a probability ϵ that the class label on a training data point has been incorrectly set. Assuming i.i.d. data, write down the error function corresponding to the negative log likelihood. Verify that the usual cross-entropy error function is obtained when $\epsilon = 0$. Note that this error function makes the model robust to incorrectly labeled data, in contrast to the usual cross-entropy error function.
- The cross-entropy error function for binary classification problems was derived for a network having a logistic-sigmoid output activation function, so that $0 \leq y(x, w) \leq 1$, and data having target values $t \in \{0, 1\}$. Derive the corresponding error function if we consider a network having an output $-1 \leq y(x, w) \leq 1$ and target values $t = 1$ for class C_1 and $t = -1$ for class C_2 . What would be the appropriate choice of output-unit activation function?

7.1 Multi-class Likelihood vs. Cross-Entropy

We are given a dataset of N independent observations. For a single data point n , let the target vector be t_n (one-hot encoded) and the model output be y_n . Since the classes are mutually exclusive, the probability of observing the target vector t_n given the input x_n follows a multinomial distribution:

$$p(t_n|x_n, w) = \prod_{k=1}^K y_k(x_n, w)^{t_{kn}}$$

The likelihood function $L(w)$ for the entire dataset is the product of the probabilities of the individual data points (assuming i.i.d.):

$$L(w) = \prod_{n=1}^N \prod_{k=1}^K y_k(x_n, w)^{t_{kn}}$$

To maximize the likelihood, it is equivalent to maximizing the log-likelihood $\ln L(w)$, because the logarithm is a monotonic function.

$$\ln L(w) = \sum_{n=1}^N \sum_{k=1}^K t_{kn} \ln y_k(x_n, w)$$

Maximizing a function is equivalent to minimizing its negative. Thus, we define the error function $E(w)$ as the negative log-likelihood:

$$E(w) = -\ln L(w) = -\sum_{n=1}^N \sum_{k=1}^K t_{kn} \ln y_k(x_n, w)$$

This is exactly the cross-entropy error function provided in the problem statement. Therefore, maximizing the likelihood is equivalent to minimizing the cross-entropy error.

7.2 Error Function with Noisy Labels

Let $y_n = p(\text{true class} = 1|x_n)$ be the output of the neural network. We are given that the label is incorrect with probability ϵ . Let $t_n \in \{0, 1\}$ be the observed label. We need to model the probability of the observed label $P(t_n = 1|x_n)$.

The observed label is 1 in two cases: 1. The true class is 1 (probability y_n) AND the label was not flipped (probability $1 - \epsilon$). 2. The true class is 0 (probability $1 - y_n$) AND the label was flipped (probability ϵ).

So, the probability that the observed label is 1 is:

$$\pi_n = P(t_n = 1|x_n) = y_n(1 - \epsilon) + (1 - y_n)\epsilon$$

Similarly, the probability that the observed label is 0 is:

$$1 - \pi_n = P(t_n = 0|x_n) = y_n\epsilon + (1 - y_n)(1 - \epsilon)$$

The likelihood for a single data point t_n is Bernoulli:

$$p(t_n|x_n) = \pi_n^{t_n} (1 - \pi_n)^{1-t_n}$$

The negative log-likelihood (error function) for the dataset is:

$$E(w) = -\sum_{n=1}^N [t_n \ln \pi_n + (1 - t_n) \ln(1 - \pi_n)]$$

Substituting π_n :

$$E(w) = - \sum_{n=1}^N [t_n \ln(y_n(1 - \epsilon) + (1 - y_n)\epsilon) + (1 - t_n) \ln(y_n\epsilon + (1 - y_n)(1 - \epsilon))]$$

Verification for $\epsilon = 0$: If $\epsilon = 0$, then $\pi_n = y_n(1) + (1 - y_n)(0) = y_n$. The error function becomes:

$$E(w) = - \sum_{n=1}^N [t_n \ln y_n + (1 - t_n) \ln(1 - y_n)]$$

This matches the standard binary cross-entropy error function.

7.3 Error Function for Targets in $\{-1, 1\}$

We want to map the network output $y \in [-1, 1]$ to a probability. A standard probability $p \in [0, 1]$ corresponds to class C_1 (target $t = 1$). We can map the range $[-1, 1]$ to $[0, 1]$ using the transformation:

$$p(C_1|x) = \frac{1 + y}{2}$$

Consequently:

$$p(C_2|x) = 1 - p(C_1|x) = 1 - \frac{1 + y}{2} = \frac{1 - y}{2}$$

The target values are $t \in \{1, -1\}$. We can combine these into a single likelihood expression. Note that when $t = 1$, we want $\frac{1+y}{2}$, and when $t = -1$, we want $\frac{1-y}{2}$.

$$P(t|x) = \left(\frac{1 + y}{2}\right)^{\frac{1+t}{2}} \left(\frac{1 - y}{2}\right)^{\frac{1-t}{2}}$$

(Note: The exponents $\frac{1+t}{2}$ and $\frac{1-t}{2}$ act as indicators for $t = 1$ and $t = -1$ respectively).

Taking the negative logarithm to find the error function:

$$E(w) = - \sum_{n=1}^N \ln \left[\left(\frac{1 + y_n}{2}\right)^{\frac{1+t_n}{2}} \left(\frac{1 - y_n}{2}\right)^{\frac{1-t_n}{2}} \right]$$

$$E(w) = - \sum_{n=1}^N \left[\frac{1+t_n}{2} \ln \left(\frac{1 + y_n}{2}\right) + \frac{1-t_n}{2} \ln \left(\frac{1 - y_n}{2}\right) \right]$$

Appropriate Activation Function: The standard logistic sigmoid $\sigma(z) = \frac{1}{1+e^{-z}}$ maps $\mathbb{R} \rightarrow [0, 1]$. For the output range $[-1, 1]$, the appropriate activation function is hyperbolic tangent (tanh):

$$y = \tanh(z) = \frac{e^z - e^{-z}}{e^z + e^{-z}}$$

This is related to the sigmoid by $\tanh(z) = 2\sigma(2z) - 1$, fitting the required range exactly.

8. Question 8: Stochastic Stability and the "Dead ReLU"

Consider a single neuron $z = w^\top x + b$ with the rectified linear activation $a = \max(0, z)$. Let $x \in \mathbb{R}^n$ and $w \in \mathbb{R}^n$ be independent random vectors whose components are i.i.d., with $x \sim \mathcal{N}(0, \sigma_x^2 I)$ and $w \sim \mathcal{N}(0, \sigma_w^2 I)$. Assume that during backpropagation the ReLU derivative is taken as $\partial a / \partial z = 1_{z>0}$ (i.e., the subgradient at $z = 0$ is 0).

- (a) **Distribution Proof.** Show that the pre-activation z follows a Gaussian distribution $\mathcal{N}(b, \sigma_z^2)$ and derive a formula for σ_z^2 in terms of n , σ_w^2 , and σ_x^2 .
- (b) **Activation Probability.** Using the standard normal cumulative distribution function Φ , derive an expression for the probability that the neuron is active, $P(a > 0)$, as a function of the bias b and the variance σ_z^2 .
- (c) **The 3σ Threshold.** For a bias initialized at $b = -3\sigma_z$, compute the numerical value of $P(a > 0)$. If a layer contains $N = 1000$ neurons initialized in this way, how many neurons are expected to be active?
- (d) **The Zero-Gradient Proof.** Show that if $z \leq 0$ for all samples in a training mini-batch, then the batch gradient with respect to the weights satisfies $\frac{\partial L}{\partial w} = 0$ and hence the weight update $\Delta w = 0$. Explain the mathematical implication of this result for "resurrecting" such a neuron during training.

8.1 1. Distribution Proof

The pre-activation z is given by the scalar product plus bias:

$$z = \sum_{i=1}^n w_i x_i + b$$

We analyze the mean and variance of the term $S = \sum_{i=1}^n w_i x_i$. Since w_i and x_i are independent and have zero mean:

$$E[w_i x_i] = E[w_i] E[x_i] = 0 \cdot 0 = 0$$

The variance of the product of two independent zero-mean random variables is the product of their variances:

$$\text{Var}(w_i x_i) = E[(w_i x_i)^2] - (E[w_i x_i])^2 = E[w_i^2] E[x_i^2] - 0 = \sigma_w^2 \sigma_x^2$$

Since z is the sum of n i.i.d. random variables $(w_i x_i)$ plus a constant b , we invoke the Central Limit Theorem (assuming n is sufficiently large). The sum of i.i.d. variables converges to a Gaussian distribution.

The mean of z :

$$E[z] = \sum_{i=1}^n E[w_i x_i] + b = 0 + b = b$$

The variance of z (since covariance terms are zero due to independence):

$$\sigma_z^2 = \text{Var}(z) = \sum_{i=1}^n \text{Var}(w_i x_i) = \sum_{i=1}^n \sigma_w^2 \sigma_x^2 = n \sigma_w^2 \sigma_x^2$$

Thus, the distribution is:

$$z \sim \mathcal{N}(b, n\sigma_w^2\sigma_x^2)$$

8.2 2. Activation Probability

The neuron is active when $a > 0$, which implies $z > 0$. Given $z \sim \mathcal{N}(b, \sigma_z^2)$, we standardize the variable using $Z = \frac{z-b}{\sigma_z} \sim \mathcal{N}(0, 1)$.

$$P(a > 0) = P(z > 0) = P\left(\frac{z-b}{\sigma_z} > \frac{0-b}{\sigma_z}\right)$$

$$P(a > 0) = P\left(Z > -\frac{b}{\sigma_z}\right)$$

Using the symmetry of the standard normal distribution ($P(Z > -x) = P(Z < x) = \Phi(x)$):

$$P(a > 0) = \Phi\left(\frac{b}{\sigma_z}\right)$$

8.3 3.

We are given $b = -3\sigma_z$. Substituting this into our probability expression:

$$P(a > 0) = \Phi\left(\frac{-3\sigma_z}{\sigma_z}\right) = \Phi(-3)$$

Using the empirical rule (68-95-99.7) or a standard normal table: The area within $\pm 3\sigma$ is approximately 0.9973. The area in the tails (outside $\pm 3\sigma$) is $1 - 0.9973 = 0.0027$. The area in the lower tail ($Z < -3$) is half of that:

$$\Phi(-3) \approx \frac{0.0027}{2} = 0.00135$$

For a layer with $N = 1000$ neurons, the expected number of active neurons is:

$$E[\text{active}] = N \times P(a > 0) = 1000 \times 0.00135 = 1.35$$

Answer: Approximately **1 neuron** is expected to be active.

8.4 4. The Zero-Gradient Proof

Let the loss function be J . The gradient with respect to the weights for a single sample is given by the chain rule:

$$\frac{\partial J}{\partial w} = \frac{\partial J}{\partial a} \cdot \frac{\partial a}{\partial z} \cdot \frac{\partial z}{\partial w}$$

The derivative of the ReLU function is:

$$\frac{\partial a}{\partial z} = \mathbb{I}(z > 0) = \begin{cases} 1 & \text{if } z > 0 \\ 0 & \text{if } z \leq 0 \end{cases}$$

If $z \leq 0$ for all samples in the mini-batch, then $\frac{\partial a}{\partial z} = 0$ for every sample. Consequently, the gradient for each sample i is zero:

$$\left(\frac{\partial J}{\partial w} \right)_i = \frac{\partial J}{\partial a_i} \cdot 0 \cdot x_i = 0$$

The batch gradient is the average (or sum) of individual gradients:

$$\nabla_w J_{\text{batch}} = \frac{1}{B} \sum_{i=1}^B \left(\frac{\partial J}{\partial w} \right)_i = 0$$

Since the gradient is zero, the gradient descent update $\Delta w = -\eta \nabla_w J$ will be exactly $\mathbf{0}$.

Mathematical Implication (The "Dying ReLU" Problem): If a neuron is initialized or updated such that it is inactive ($z \leq 0$) for the entire dataset (or batch), its weights will never update because the gradient signal is blocked by the zero derivative of the ReLU. Without weight updates, the distribution of z remains fixed (assuming fixed inputs). Thus, the neuron is permanently "dead" and will never learn or contribute to the network's function.

9. Question 9: Variance Preservation and Xavier Initialization

Assume a linear neural network of depth L where each layer is defined as $y^{(l)} = W^{(l)} x^{(l)}$. The weights $W_{ij}^{(l)}$ initialized i.i.d. with mean 0 and variance $\text{Var}(W)$.

- Symmetric Constraints:** Prove that to maintain constant signal variance in the forward pass, we require $\text{Var}(W) = 1/n_{in}$ and to maintain constant gradient variance in the backward pass, we require $\text{Var}(W) = 1/n_{out}$.
- The Glorot Synthesis:** Derive the Xavier variance $\sigma^2 = \frac{2}{n_{in} + n_{out}}$ as a compromise between these two constraints.
- Depth-Induced Variance:** Let the layer gain be $G = n \text{Var}(W)$. Derive the expression for the output variance of the L -th layer, $\text{Var}(y^{(L)})$, as a function of the initial input variance $\text{Var}(x^{(0)})$.
- The Stability Bound:** Suppose the initialization contains a small error ϵ such that $n \cdot \text{Var}(W) = 1 + \epsilon$. Show that the output variance grows exponentially with depth L .

- (e) Numerical Analysis: Compute the amplification factor for a network where $L = 150$ and $\epsilon = 0.02$. Discuss why this exponential growth poses a significant risk for training in 16-bit floating-point (FP16) precision.

9.1 1. Symmetric Constraints

Forward Pass: Consider a single layer $y = Wx$. The output y_i is given by $y_i = \sum_{j=1}^{n_{in}} W_{ij}x_j$. Assuming x and W are independent and have zero means:

$$\text{Var}(y_i) = \text{Var}\left(\sum_{j=1}^{n_{in}} W_{ij}x_j\right) = \sum_{j=1}^{n_{in}} \text{Var}(W_{ij}x_j)$$

Using the property $\text{Var}(AB) = \text{Var}(A)\text{Var}(B) + E[A]^2\text{Var}(B) + E[B]^2\text{Var}(A)$. Since means are zero:

$$\text{Var}(y_i) = \sum_{j=1}^{n_{in}} \text{Var}(W)\text{Var}(x_j) = n_{in} \text{Var}(W)\text{Var}(x)$$

To maintain constant variance ($\text{Var}(y) = \text{Var}(x)$):

$$n_{in} \text{Var}(W) = 1 \implies \text{Var}(W) = \frac{1}{n_{in}}$$

Backward Pass: The gradient propagates as $\frac{\partial L}{\partial x} = W^\top \frac{\partial L}{\partial y}$. Let $\delta^{(l)} = \frac{\partial L}{\partial y^{(l)}}$. Then $\delta^{(l-1)} = W^\top \delta^{(l)}$. The dimensionality changes from n_{out} to n_{in} , summing over n_{out} terms:

$$\text{Var}(\delta_i^{(l-1)}) = n_{out} \text{Var}(W)\text{Var}(\delta^{(l)})$$

To maintain constant gradient variance:

$$n_{out} \text{Var}(W) = 1 \implies \text{Var}(W) = \frac{1}{n_{out}}$$

9.2 The Glorot Synthesis

We have two conflicting constraints: $\text{Var}(W) = \frac{1}{n_{in}}$ and $\text{Var}(W) = \frac{1}{n_{out}}$. Xavier Glorot proposed using the harmonic mean of the two constraints to balance them. Alternatively, we can view it as averaging the constraints $n_{in} \text{Var}(W) + n_{out} \text{Var}(W) \approx 2$ (aiming for sum of gains to be 2 to average to 1? A simpler arithmetic average of the limits is standard). Commonly derived by setting the variance to:

$$\text{Var}(W) = \frac{2}{n_{in} + n_{out}}$$

This is the harmonic mean of the two bounds (with a scaling factor to average the variances properly). It ensures that the variance is not too large for the forward pass nor too small for the backward pass.

9.3 Depth-Induced Variance

Let the gain of layer l be $G_l = \frac{\text{Var}(y^{(l)})}{\text{Var}(y^{(l-1)})} = n_{in} \text{Var}(W)$. Assuming the width n and $\text{Var}(W)$ are constant across all L layers, then $G_l = G = n \text{Var}(W)$ for all l . The output variance after L layers is the product of the per-layer amplifications:

$$\text{Var}(y^{(L)}) = G \cdot \text{Var}(y^{(L-1)}) = G^2 \cdot \text{Var}(y^{(L-2)}) = \dots = G^L \cdot \text{Var}(x^{(0)})$$

Substituting G :

$$\text{Var}(y^{(L)}) = (n \text{Var}(W))^L \text{Var}(x^{(0)})$$

9.4 The Stability Bound

We are given $n \cdot \text{Var}(W) = 1 + \epsilon$. Substituting this into the result from Part 3:

$$\text{Var}(y^{(L)}) = (1 + \epsilon)^L \text{Var}(x^{(0)})$$

For small ϵ , we can approximate $(1 + \epsilon)^L$ using the limit definition of the exponential function $e^x = \lim_{n \rightarrow \infty} (1 + x/n)^n$. Or simply by observation, a geometric progression $(1 + \epsilon)^L$ grows exponentially with L . Specifically, if $\epsilon > 0$, the variance explodes. If $\epsilon < 0$, the variance vanishes.

9.5 Numerical Analysis

Given $L = 150$ and $\epsilon = 0.02$. The amplification factor is:

$$A = (1 + 0.02)^{150} = (1.02)^{150}$$

Calculation:

$$(1.02)^{150} \approx 19.46$$

So the variance is amplified by nearly 20 times.

Risk for FP16 Training: FP16 (half-precision) has a limited dynamic range. - Max value $\approx 65,504$. - Smallest positive normal number $\approx 6 \times 10^{-5}$. While a factor of 20 seems manageable, this is only the variance (standard deviation scales by $\sqrt{20} \approx 4.5$). However, if ϵ were slightly larger, or the network deeper, or if we consider gradients (which effectively double the depth $L \rightarrow 2L$ for round trip), the values can easily overflow (explode to infinity) or underflow (vanish to zero). Specifically, if we looked at the gradient signal, an error of ϵ in the other direction could cause gradients to vanish by a factor of $1/20$, losing precision. More critically, if the initialization variance is slightly off in the other direction $(1 - \epsilon)$, signal vanishes: $(0.98)^{150} \approx 0.048$, pushing activations towards zero where precision in FP16 is lower (denormalized numbers).

10. Question 10: Transfer Learning with Pre-trained Networks

10.1 Results

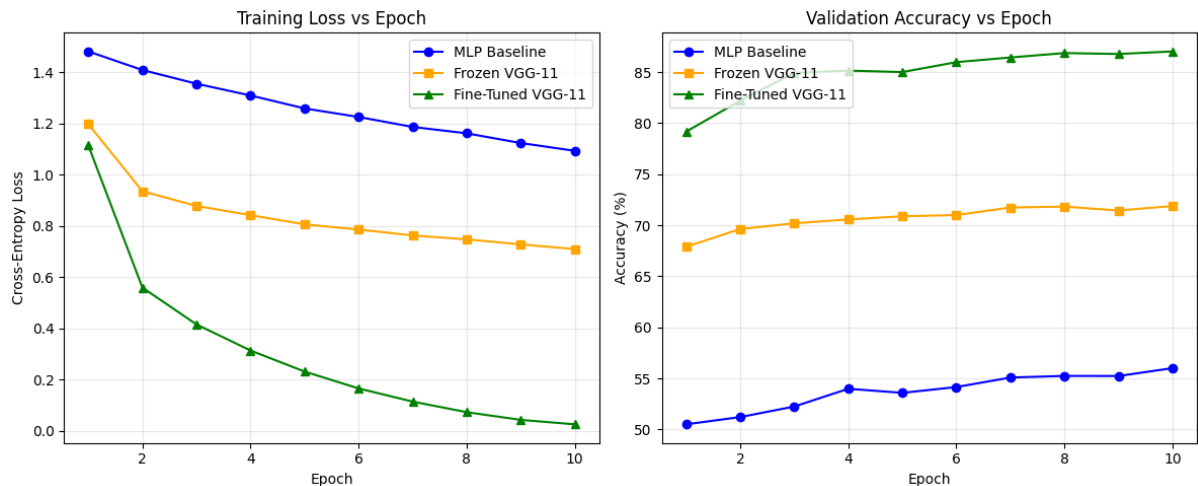


Figure 4: Training Cross-Entropy Loss (left) and Validation Accuracy (right) over 10 epochs for the MLP Baseline (blue), Frozen VGG-11 (orange), and Fine-Tuned VGG-11 (green).

10.2 Discussion

Based on the training curves shown above, we address the discussion points regarding convergence and generalization:

10.2.1 Convergence and Stability

- **Speed and Smoothness:** The MLP Baseline (blue line) exhibits the slowest convergence, with a shallow, linear decrease in loss and only reaching $\sim 55\%$ accuracy. In contrast, the Fine-Tuned VGG-11 (green line) shows extremely rapid convergence, with the loss dropping below 0.6 within just two epochs and accuracy exceeding 80% almost immediately. The curves for the pre-trained models are significantly smoother and steeper than the scratch-trained MLP.
- **Role of Pre-trained Initialization:** Pre-trained initialization leads to faster and more stable optimization because the network weights are already initialized in a favorable region of the loss landscape (a "good basin"). The network has already learned robust feature detectors (edges, textures, shapes) from ImageNet. Consequently, the optimization process does not need to learn these fundamental representations from scratch; it only needs to adapt the high-level features to the specific classes of CIFAR-10, resulting in a shorter and more direct path to the global minimum.

10.2.2 Generalization Gap and Spatial Inductive Bias

- **Performance Gap:** Despite having the vast majority of its weights frozen, the Frozen VGG-11 (orange line) significantly outperforms the fully trainable MLP Baseline (reaching $\sim 72\%$ vs. $\sim 56\%$ accuracy).
- **Spatial Inductive Bias:** This gap is explained by the spatial inductive bias inherent in Convolutional Neural Networks (CNNs). The MLP treats the input as a flattened vector of 3,072 features, discarding all 2D spatial structure (neighborhood relationships between pixels). It must relearn that adjacent pixels are related.
- Conversely, the VGG architecture assumes *locality* (via convolutions) and *translation invariance* (via pooling). Even with frozen weights, the convolutional layers effectively transform raw pixels into semantically meaningful feature maps that preserve spatial hierarchy. This provides the final linear classifier with high-quality, linearly separable representations, whereas the MLP struggles to extract these features from the raw, flattened data.